

```
In [1]: #importing Libraries

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
```

```
In [2]: #importing Data set
hd =pd.read_csv(r"C:\Users\sakr\OneDrive\Desktop\Intern\Heart Disease Analysis\Heart Disease data.csv")
hd
```

```
Out[2]:
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3	0
1	53	1	0	140	203	1	0	155	1	3.1	0	0	3	0
2	70	1	0	145	174	0	1	125	1	2.6	0	0	3	0
3	61	1	0	148	203	0	1	161	0	0.0	2	1	3	0
4	62	0	0	138	294	1	1	106	0	1.9	1	3	2	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1020	59	1	1	140	221	0	1	164	1	0.0	2	0	2	1
1021	60	1	0	125	258	0	0	141	1	2.8	1	1	3	0
1022	47	1	0	110	275	0	0	118	1	1.0	1	1	2	0
1023	50	0	0	110	254	0	0	159	0	0.0	2	0	2	1
1024	54	1	0	120	188	0	1	113	0	1.4	1	1	3	0

1025 rows × 14 columns

```
In [3]: # Null Value count

hd.isna().sum()
```

```
Out[3]: age      0
sex      0
cp       0
trestbps 0
chol     0
fbs      0
restecg  0
thalach  0
exang    0
oldpeak  0
slope    0
ca       0
thal     0
target   0
dtype: int64
```

```
In [4]: #Column names

# Attribute Information:

# age
# sex
# chest pain type (4 values)
# resting blood pressure
# serum cholestoral in mg/dl
# fasting blood sugar > 120 mg/dl
# resting electrocardiographic results (values 0,1,2)
# maximum heart rate achieved
# exercise induced angina
# oldpeak = ST depression induced by exercise relative to rest
# the slope of the peak exercise ST segment
# number of major vessels (0-3) colored by flourosopy
# thal: 0 = normal; 1 = fixed defect; 2 = reversable defect
# target= affected people 0=no,1=yes

hd.columns.to_list()
```

```
Out[4]: ['age',  
        'sex',  
        'cp',  
        'trestbps',  
        'chol',  
        'fbs',  
        'restecg',  
        'thalach',  
        'exang',  
        'oldpeak',  
        'slope',  
        'ca',  
        'thal',  
        'target']
```

```
In [5]: #Adding new columns  
  
def check_disease(df):  
    if df["target"] == 1:  
        return "Disease"  
    else:  
        return "No Disease"  
  
hd["Heart Disease"] = hd.apply(check_disease, axis = 1)  
  
def check_gender(df):  
    if df["sex"] == 1:  
        return "Male"  
    elif df["sex"] == 0:  
        return "Female"  
    else:  
        return "Unknown"  
  
hd["Gender"] = hd.apply(check_gender, axis = 1)  
  
def check_CP(df):  
    if df["cp"] == 0:  
        return "Typical Angina"  
    elif df["cp"] == 1:  
        return "Atypical Angina"  
    elif df["cp"] == 2:  
        return "Non-Anginal Pain"  
    elif df["cp"] == 3:  
        return "Asymptomatic"
```

```
    else:
        return "Unknown"

hd["CP_Description"] = hd.apply(check_CP, axis = 1)

def check_fbs(df):
    if df["fbs"] == 1:
        return ">125 mg/dl"
    elif df['fbs'] == 0:
        return "<125 mg/dl"
    else:
        return "Unknown"

hd["FBS > 125"] = hd.apply(check_fbs, axis = 1)

def check_thal(df):
    if df["thal"] == 0:
        return "normal"
    elif df['thal'] == 1:
        return "fixed defect"
    elif df['thal'] == 2:
        return "reversible defect"
    else:
        return "Unknown"

hd["Thallium"] = hd.apply(check_thal, axis =1)

def check_slope(df):
    if df["slope"] == 0:
        return "upsloping"
    elif df['slope'] == 1:
        return "flat"
    elif df['slope'] == 2:
        return "downsloping"
    else:
        return "Unknown"

hd["Slope Description"] = hd.apply(check_slope, axis =1)

def check_exang(df):
    if df["exang"] == 0:
        return "no"
    elif df['slope'] == 1:
        return "yes"
```

```
        else:
            return "Unknown"

hd["Exerscised Induced Angina"] = hd.apply(check_exang, axis = 1)

def check_restecg(df):
    if df["restecg"] == 0:
        return "normal"
    elif df['slope'] == 1:
        return "having ST-T wave abnormality (T wave inversions and/or ST elevation or depression of > 0.05 mV)"
    elif df['slope'] == 2:
        return "showing probable or definite left ventricular hypertrophy by Estes' criteria"
    else:
        return "Unknown"

hd["Rest ECG Description"] = hd.apply(check_restecg, axis = 1)

hd
```

Out[5]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	...	thal	target	Heart Disease	Gender	CP_Description	FBS > 125	Thallium	SI Descrip
0	52	1	0	125	212	0	1	168	0	1.0	...	3	0	No Disease	Male	Typical Angina	<125 mg/dl	Unknown	downslope
1	53	1	0	140	203	1	0	155	1	3.1	...	3	0	No Disease	Male	Typical Angina	>125 mg/dl	Unknown	upslope
2	70	1	0	145	174	0	1	125	1	2.6	...	3	0	No Disease	Male	Typical Angina	<125 mg/dl	Unknown	upslope
3	61	1	0	148	203	0	1	161	0	0.0	...	3	0	No Disease	Male	Typical Angina	<125 mg/dl	Unknown	downslope
4	62	0	0	138	294	1	1	106	0	1.9	...	2	0	No Disease	Female	Typical Angina	>125 mg/dl	reversible defect	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1020	59	1	1	140	221	0	1	164	1	0.0	...	2	1	Disease	Male	Atypical Angina	<125 mg/dl	reversible defect	downslope
1021	60	1	0	125	258	0	0	141	1	2.8	...	3	0	No Disease	Male	Typical Angina	<125 mg/dl	Unknown	
1022	47	1	0	110	275	0	0	118	1	1.0	...	2	0	No Disease	Male	Typical Angina	<125 mg/dl	reversible defect	
1023	50	0	0	110	254	0	0	159	0	0.0	...	2	1	Disease	Female	Typical Angina	<125 mg/dl	reversible defect	downslope

age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	...	thal	target	Heart Disease	Gender	CP_Description	FBS > 125	Thallium	SI Descrip
1024	54	1	0	120	188	0	1	113	0	1.4	...	3	0	No Disease	Male	Typical Angina	<125 mg/dl	Unknown

1025 rows x 22 columns

```
In [6]: hd.head()
```

Out[6]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	...	thal	target	Heart Disease	Gender	CP_Description	FBS > 125	Thallium	Slope Description
0	52	1	0	125	212	0	1	168	0	1.0	...	3	0	No Disease	Male	Typical Angina	<125 mg/dl	Unknown	downsloping
1	53	1	0	140	203	1	0	155	1	3.1	...	3	0	No Disease	Male	Typical Angina	>125 mg/dl	Unknown	upsloping
2	70	1	0	145	174	0	1	125	1	2.6	...	3	0	No Disease	Male	Typical Angina	<125 mg/dl	Unknown	upsloping
3	61	1	0	148	203	0	1	161	0	0.0	...	3	0	No Disease	Male	Typical Angina	<125 mg/dl	Unknown	downsloping
4	62	0	0	138	294	1	1	106	0	1.9	...	2	0	No Disease	Female	Typical Angina	>125 mg/dl	reversible defect	flat

5 rows × 22 columns



```
In [7]: hd[['age', 'trestbps', 'chol', 'thalach']].describe()
```


Out[7]:

	age	trestbps	chol	thalach
count	1025.000000	1025.000000	1025.000000	1025.000000
mean	54.434146	131.611707	246.000000	149.114146
std	9.072290	17.516718	51.59251	23.005724
min	29.000000	94.000000	126.000000	71.000000
25%	48.000000	120.000000	211.000000	132.000000
50%	56.000000	130.000000	240.000000	152.000000
75%	61.000000	140.000000	275.000000	166.000000
max	77.000000	200.000000	564.000000	202.000000

In [8]: *#Coreation between the variables*

```
hd[['age', 'trestbps', 'chol', 'thalach']].corr()
```

Out[8]:

	age	trestbps	chol	thalach
age	1.000000	0.271121	0.219823	-0.390227
trestbps	0.271121	1.000000	0.127977	-0.039264
chol	0.219823	0.127977	1.000000	-0.021772
thalach	-0.390227	-0.039264	-0.021772	1.000000

In [9]: *# Calculate the counts for each sex and confirmed patients*

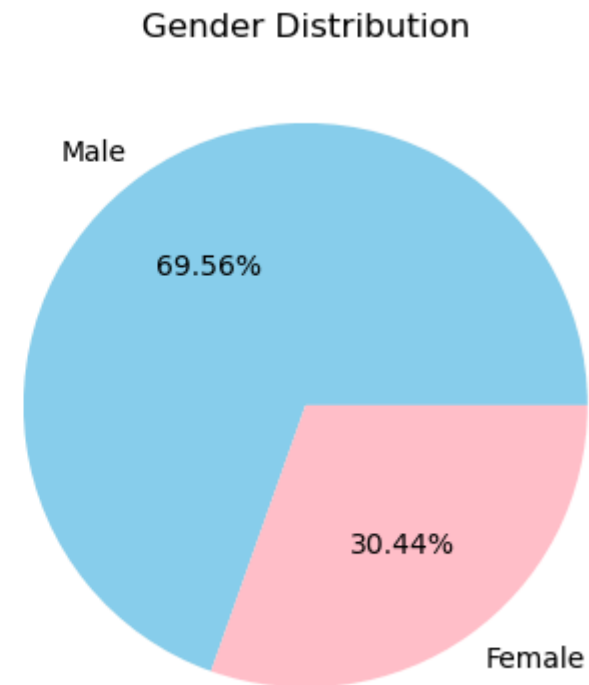
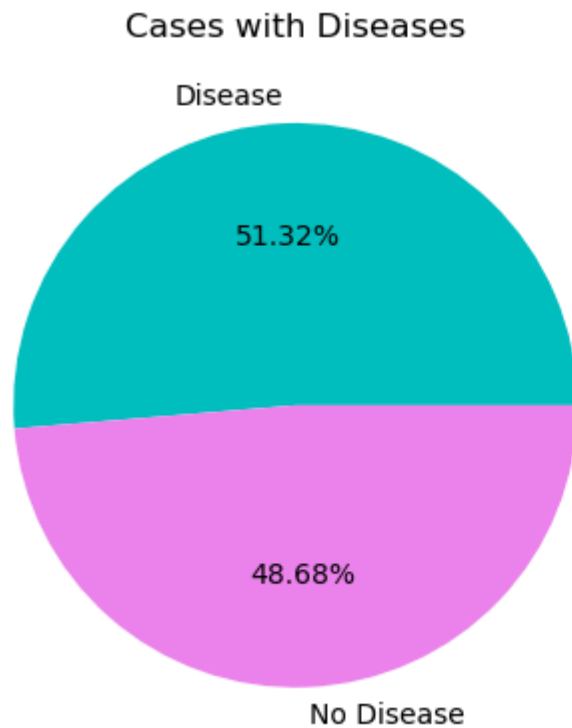
```
disease_count= hd['Heart Disease'].value_counts()
gender_counts = hd['Gender'].value_counts()
```

```
# creating plot area
plt.figure(figsize=(10,6))
```

```
#Deviding plot area and plotting confirmed cases
```

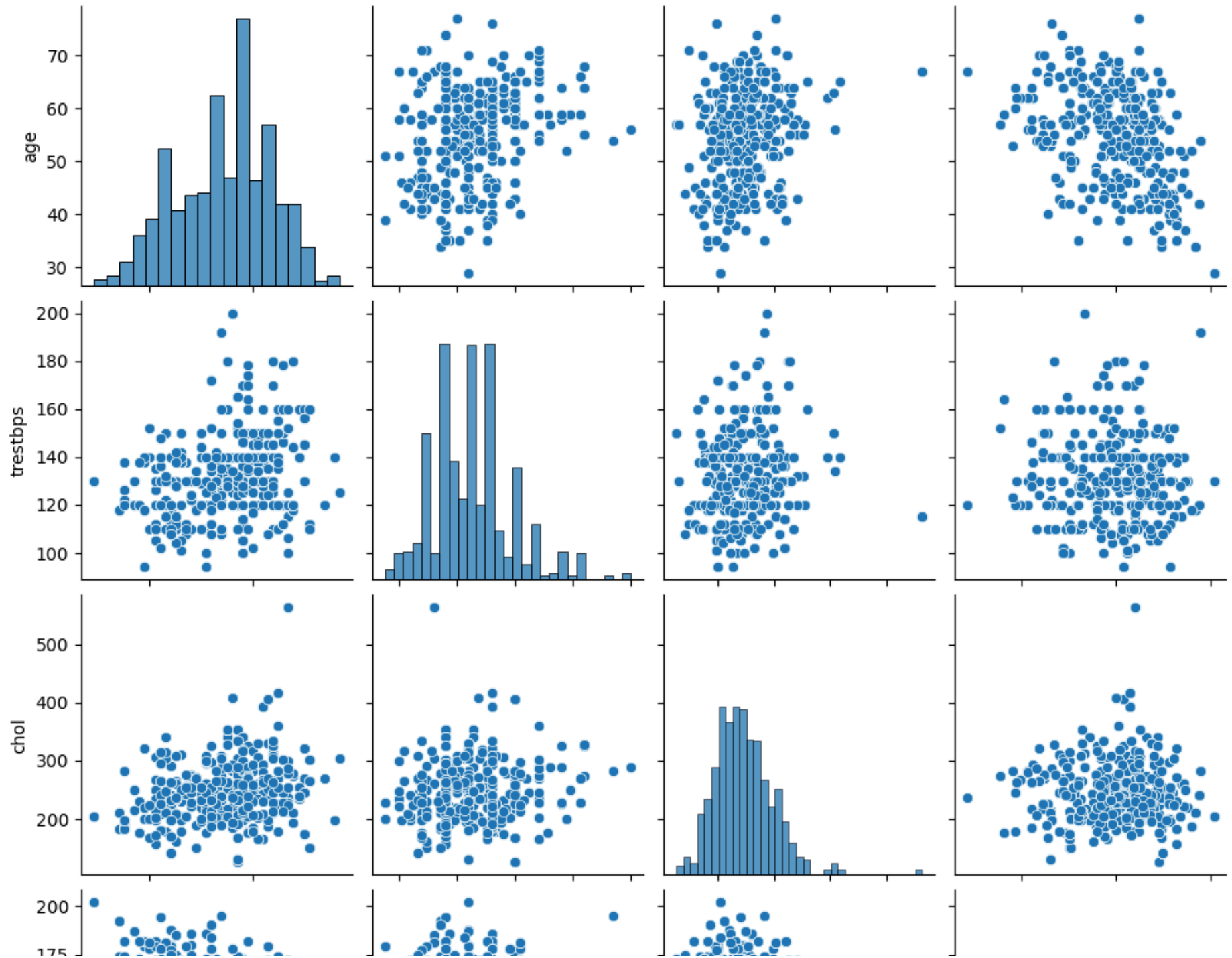
```
plt.subplot(1,2,1)
plt.pie(disease_count, labels = disease_count.index, autopct='%1.2f%%', colors = ['c', 'violet'])
plt.title('Cases with Diseases')
```

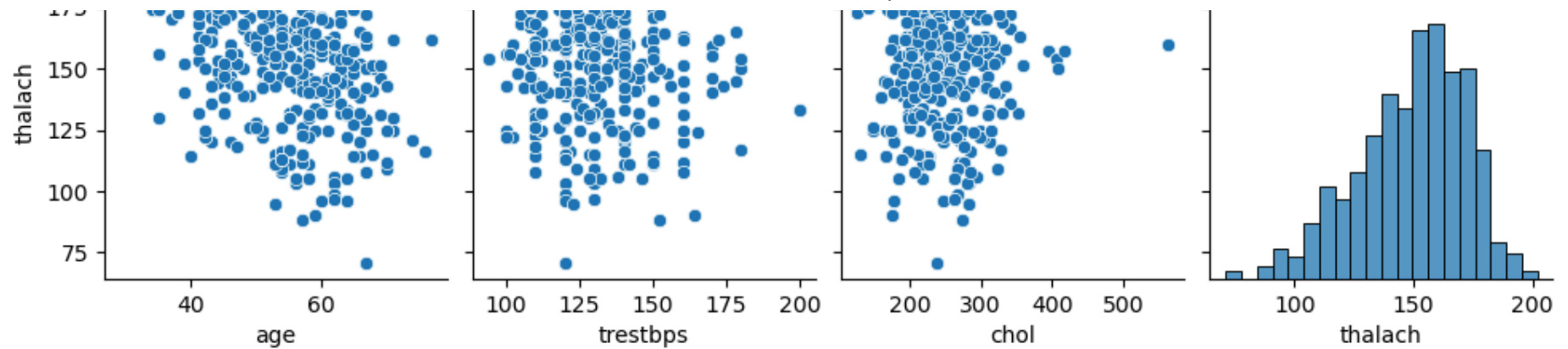
```
#Deviding plot area and showing Gender Distribution  
plt.subplot(1,2,2)  
plt.pie(gender_counts, labels = gender_counts.index, autopct='%1.2f%%', colors = ['skyblue', 'pink'])  
plt.title('Gender Distribution')  
plt.show()
```



```
In [10]: #pair plot chart showing correlation and histogram (data distribution shape)  
sns.pairplot(hd[['age', 'trestbps', 'chol', 'thalach']])
```

```
Out[10]: <seaborn.axisgrid.PairGrid at 0x1e2d147e3b0>
```





```
In [11]: #Creating plot area

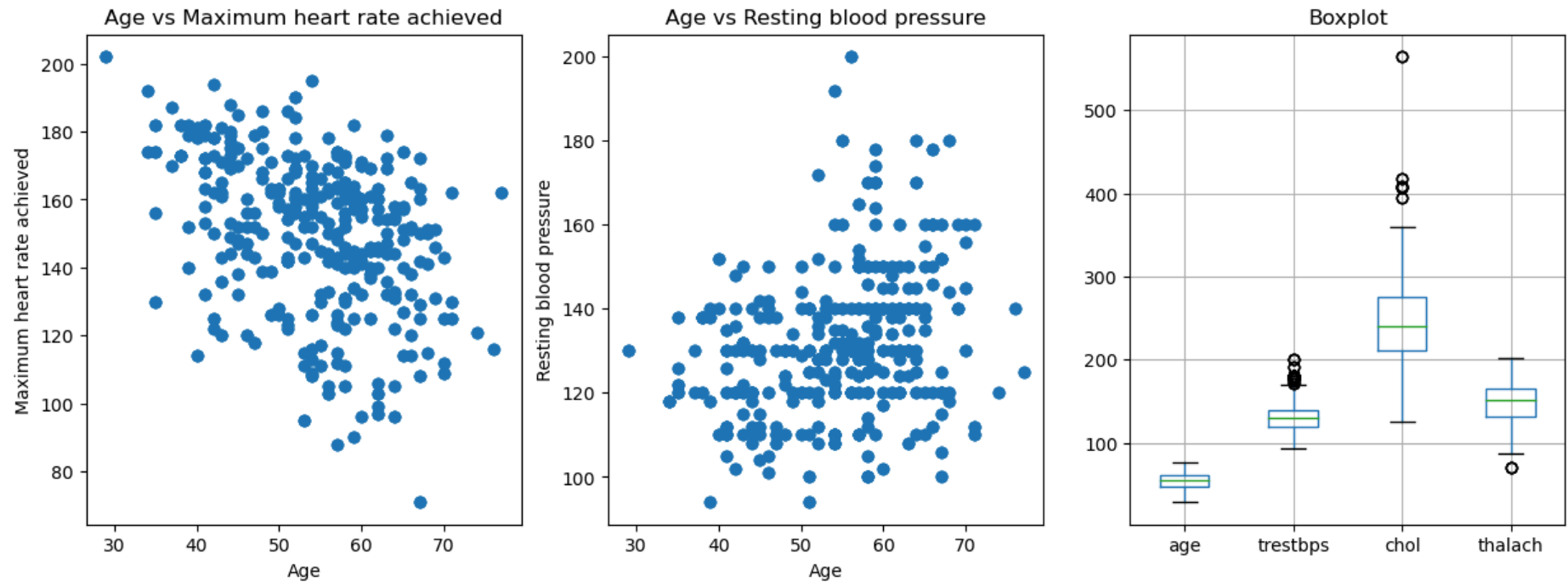
plt.figure(figsize=(15,5))

#dividing plot area and Maximum heart rate achieved
plt.subplot(1,3,1)
plt.scatter(hd['age'],hd['thalach'])
plt.xlabel('Age')
plt.ylabel('Maximum heart rate achieved')
plt.title('Age vs Maximum heart rate achieved')

#Dividing plot area and Resting blood pressure

plt.subplot(1,3,2)
plt.scatter(hd['age'],hd['trestbps'])
plt.xlabel('Age')
plt.ylabel('Resting blood pressure')
plt.title('Age vs Resting blood pressure')

# Boxplot to see outliers
plt.subplot(1,3,3)
hd[['age','trestbps','chol','thalach']].boxplot()
plt.title('Boxplot')
plt.show()
```

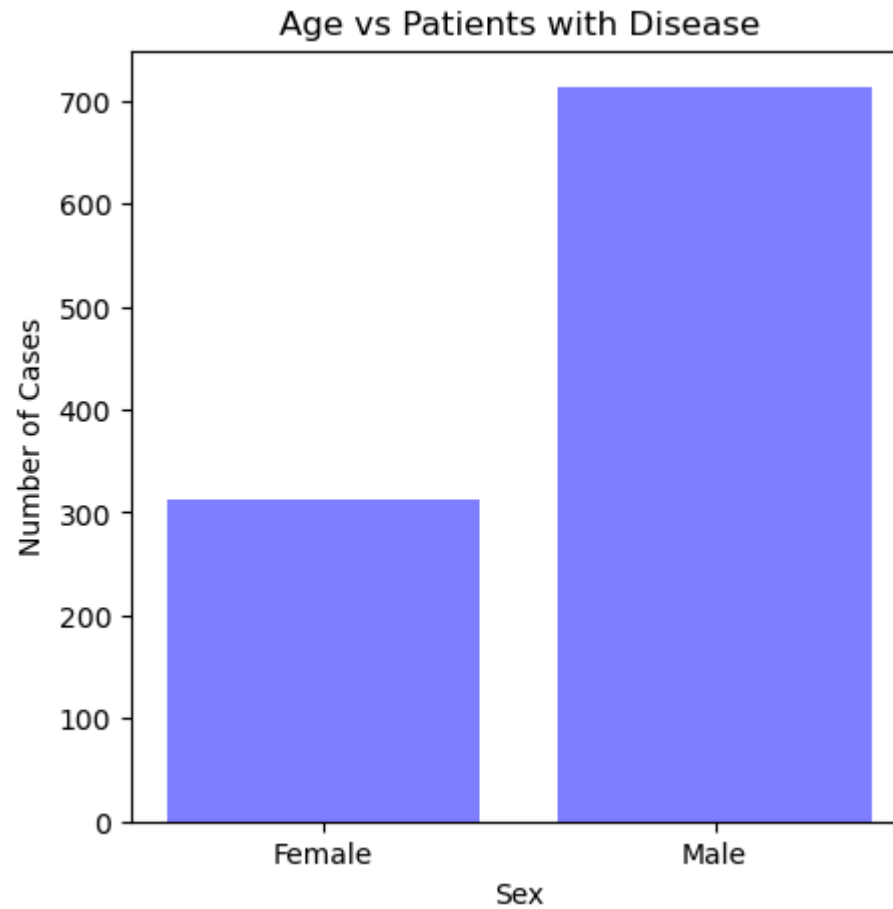


```
In [12]: plt.figure(figsize=(5,5))

# Deviving the chart area Age vs Confirmed Case

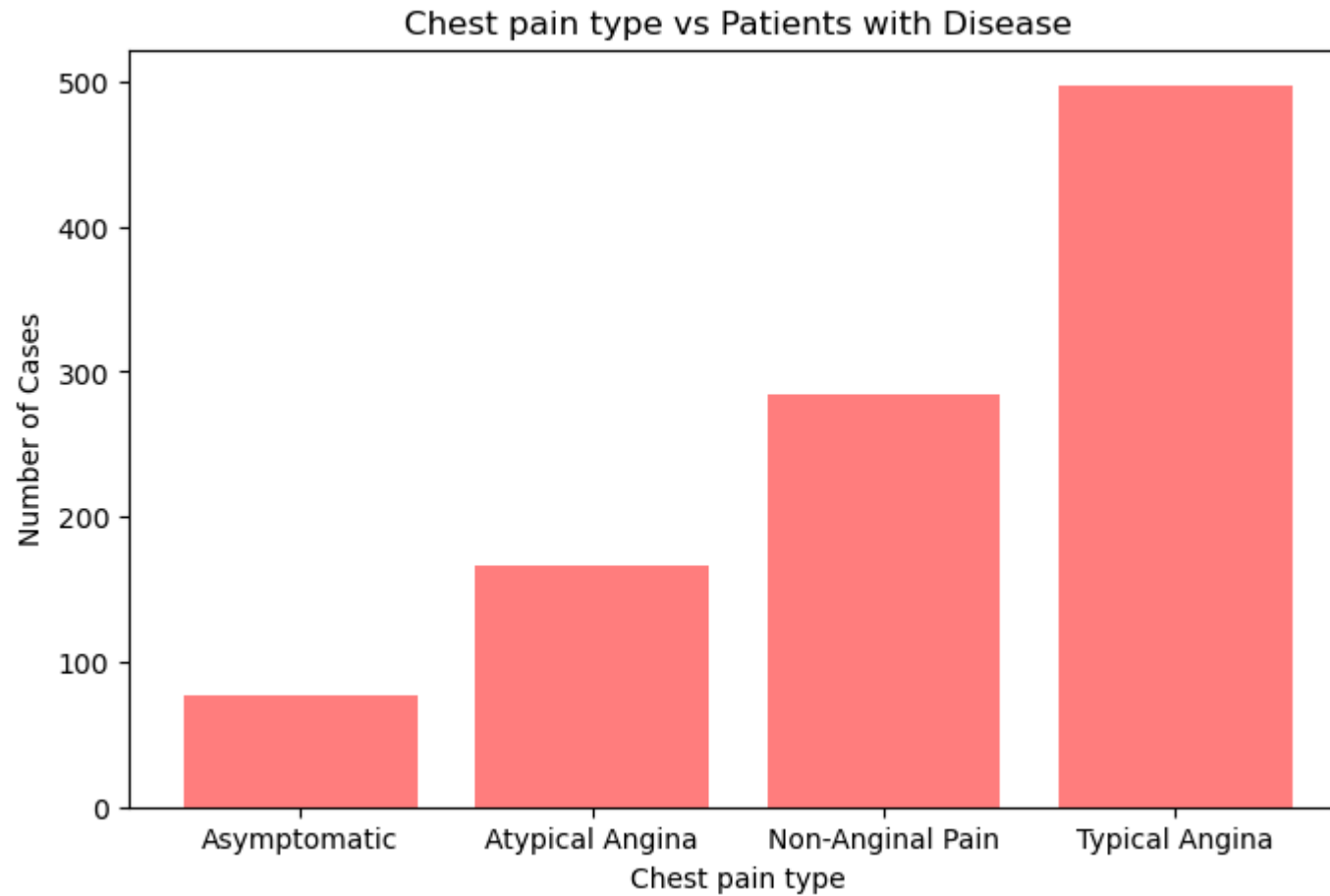
b=hd.groupby('Gender')['target'].count()
plt.bar(b.index,b.values,color='b',alpha=.5)
plt.xlabel('Sex')
plt.ylabel('Number of Cases')
plt.title('Age vs Patients with Disease')
```

```
Out[12]: Text(0.5, 1.0, 'Age vs Patients with Disease')
```



```
In [13]: plt.figure(figsize=(8,5))  
  
# Deviving the chart area Chest pain type vs Confiremd Case  
  
c=hd.groupby('CP_Description')['target'].count()  
plt.bar(c.index,c.values,color='r',alpha=.5)  
plt.xlabel('Chest pain type')  
plt.ylabel('Number of Cases')  
plt.title('Chest pain type vs Patients with Disease')
```

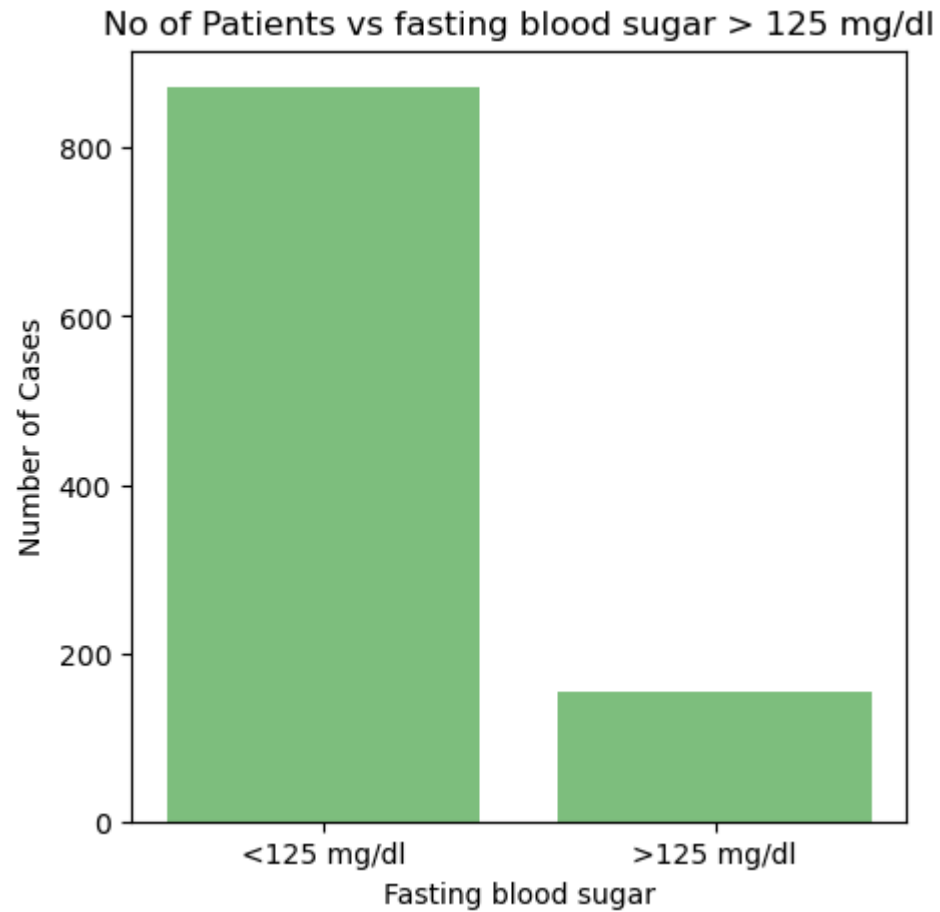
```
Out[13]: Text(0.5, 1.0, 'Chest pain type vs Patients with Disease')
```



```
In [14]: plt.figure(figsize=(5,5))

# Deviving the chart area Chest pain type vs fasting blood sugar > 120 mg/dl

d=hd.groupby('FBS > 125')['target'].count()
plt.bar(d.index,d.values,color='g',alpha=.5)
plt.xlabel('Fasting blood sugar')
plt.ylabel('Number of Cases')
plt.title('No of Patients vs fasting blood sugar > 125 mg/dl')
plt.show()
```

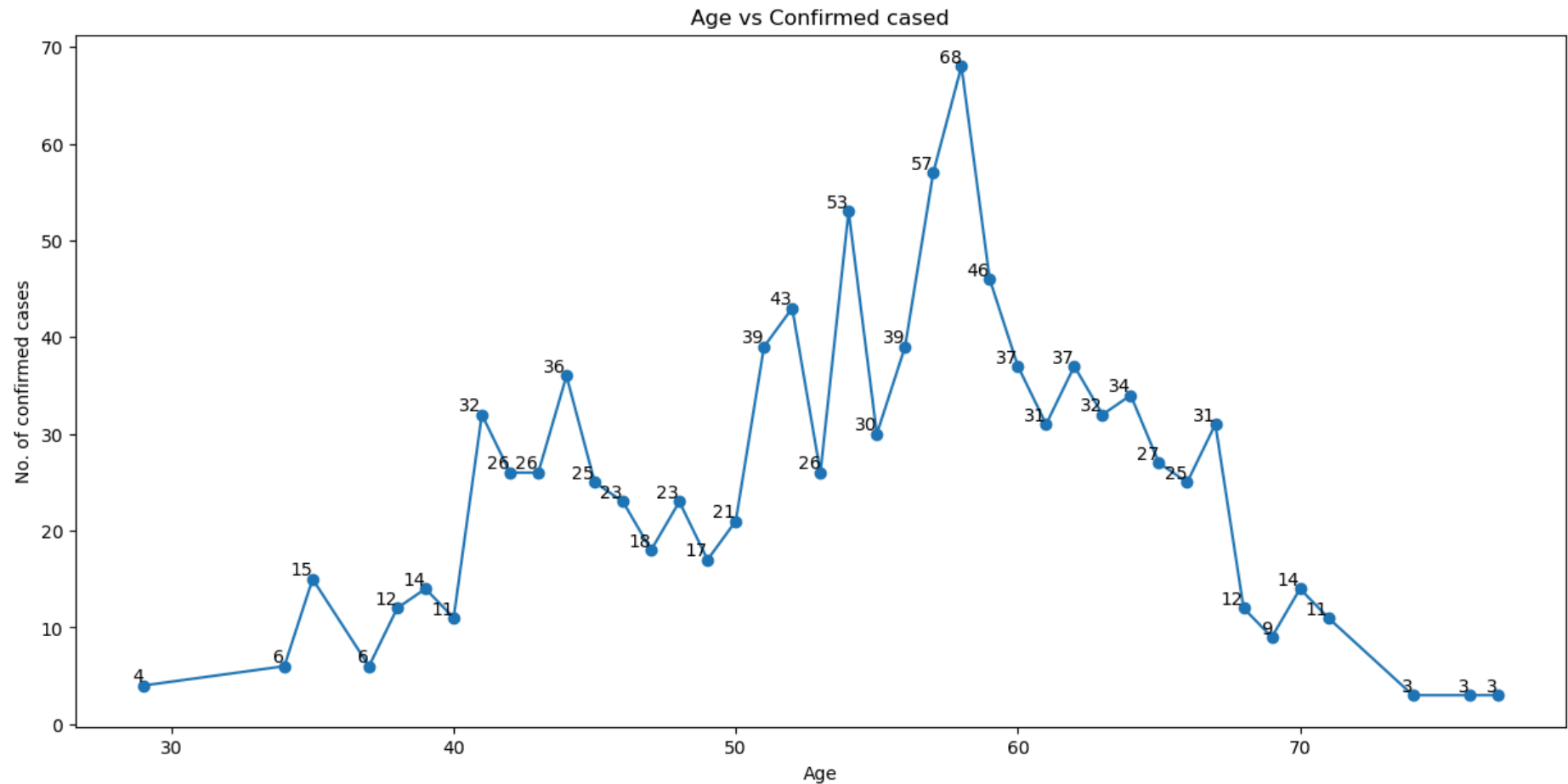


In [15]: *#creating a line chart to see age-wise confirmed cases*

```
a = hd.groupby('age')['target'].count()
plt.figure(figsize=(15,7))
plt.plot(a, marker='o')
plt.xlabel('Age')
plt.ylabel('No. of confirmed cases')
plt.title('Age vs Confirmed cases')

# Adding labels to the data points
for x, y in zip(a.index, a.values):
    plt.text(x, y, str(y), ha='right', va='bottom') # Adjust 'ha' and 'va' for label positioning

plt.show()
```

Applying ML models to make some Predictions

```
In [16]: pip install scikit-learn
```

```
Requirement already satisfied: scikit-learn in c:\users\saiqr\anaconda\lib\site-packages (1.2.1)  
Requirement already satisfied: threadpoolctl>=2.0.0 in c:\users\saiqr\anaconda\lib\site-packages (from scikit-learn) (2.2.0)  
Requirement already satisfied: numpy>=1.17.3 in c:\users\saiqr\anaconda\lib\site-packages (from scikit-learn) (1.23.5)  
Requirement already satisfied: scipy>=1.3.2 in c:\users\saiqr\anaconda\lib\site-packages (from scikit-learn) (1.10.0)  
Requirement already satisfied: joblib>=1.1.1 in c:\users\saiqr\anaconda\lib\site-packages (from scikit-learn) (1.1.1)  
Note: you may need to restart the kernel to use updated packages.
```

```
In [17]: # dataframe for machine learning
df = hd[hd.columns.to_list()]
```

```
In [18]: from sklearn.model_selection import train_test_split

# Features and target
X = df[['chol', 'trestbps']]
y = df['target']

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
In [19]: from sklearn.ensemble import RandomForestClassifier

# Initialize the model
model = RandomForestClassifier(random_state=42)

# Train the model
model.fit(X_train, y_train)
```

```
Out[19]: ▼ RandomForestClassifier
RandomForestClassifier(random_state=42)
```

```
In [20]: # Make predictions
y_pred = model.predict(X_test)
```

```
In [21]: y_pred
```

```
Out[21]: array([1, 0, 0, 1, 0, 1, 0, 0, 1, 0, 1, 0, 1, 1, 0, 0, 0, 1, 1, 0, 1, 1,
        0, 0, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0,
        1, 0, 0, 1, 0, 0, 0, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0,
        0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0,
        0, 0, 1, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 0,
        1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1,
        0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 0,
        1, 0, 0, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0, 0, 0, 0, 1, 0, 0,
        1, 1, 1, 1, 1, 0, 0], dtype=int64)
```

In [22]: `from sklearn.metrics import accuracy_score, classification_report`

```
accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy: {accuracy:.2f}')

print(classification_report(y_test, y_pred))
```

Accuracy: 0.95

	precision	recall	f1-score	support
0	0.92	0.99	0.95	102
1	0.99	0.91	0.95	103
accuracy			0.95	205
macro avg	0.95	0.95	0.95	205
weighted avg	0.95	0.95	0.95	205

We had created Random Forest Classifier model on based on three columns " Blood Pressure,Cholestrol and Age". Using that model we can Predict whether a Patient is suffering with Heart Disease or Not

```
In [24]: input_string = input("Enter Cholestrol and BP of Patient sepeated by Space: ")
values_list = list(map(int, input_string.split()))
pred_target = model.predict([values_list])
if pred_target == 1:
    print("Patient is suffering with Heart Disease")
else:
    print("Patiennt is not suffering with Heart Disease")
```

Enter Cholestrol and BP of Patient sepeated by Space: 300 150
Patient is suffering with Heart Disease

In []: